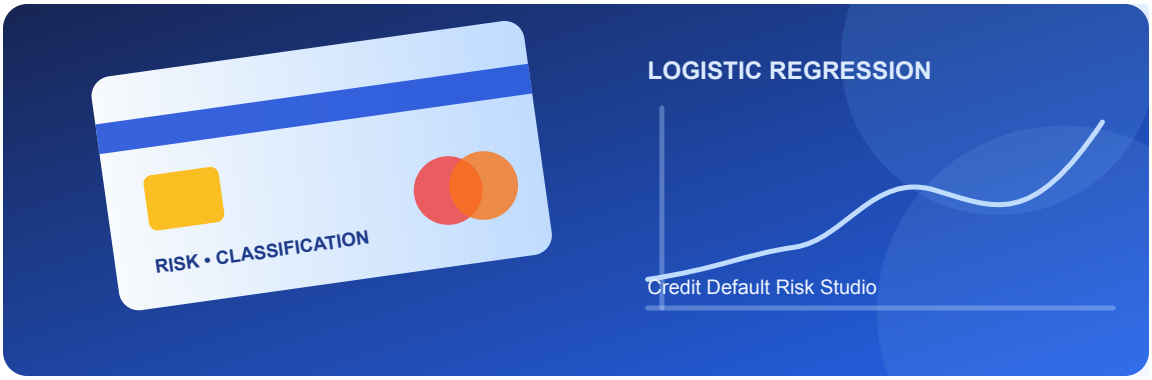
💳 Credit Default Risk Prediction



Logistic Regression · Binary Classification

Project Objective

Project Objective

Predict whether a credit-card client will default on payment in the following month using historical client data.

Algorithm

Logistic Regression — Binary Classification

Dataset

UCI Default of Credit Card Clients Dataset

Tools

- Python
- pandas and NumPy
- Matplotlib and Seaborn
- scikit-learn
- Google Colab

Workflow

1. Understand the dataset
2. Perform exploratory data analysis
3. Prepare the features and target
4. Train Logistic Regression
5. Evaluate the model
6. Interpret the results
7. Prepare the model for deployment

🧠 Logistic Regression: theory and mathematics

Logistic Regression is a supervised classification algorithm that converts a linear score into a probability. For features $x_1, \ldots, x_p$ and coefficients $\beta_1, \ldots, \beta_p$, the linear score is

$$z = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = \beta_0 + \boldsymbol{\beta}^{\mathsf{T}} \mathbf{x}.$$

The **sigmoid** (logistic) function maps any real score to $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

We interpret $\hat{p} = \sigma(z)$ as the estimated probability of the positive class. A threshold τ turns probability into a decision: predict positive when $\hat{p} \geq \tau$ and negative otherwise. The common $\tau = 0.5$ is only a default; the correct value depends on error costs, capacity and policy.

The model is linear in the **log-odds**:

$$\log\left(\frac{p}{1-p}\right) = z.$$

A one-unit increase in x_j changes the log-odds by β_j while other features are held fixed. The odds ratio is e^{β_j}: values above 1 increase estimated odds, values below 1 decrease them. This is an association, not proof of causation.

Binary cross-entropy loss

For labels $y_i \in \{0, 1\}$ and predictions $\hat{p}_i$, training minimises the average negative log-likelihood:

$$\mathcal{L}(\boldsymbol{\beta}) = -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i) \right].$$

The loss heavily penalises confident wrong predictions. Regularisation controls coefficient size. L2 regularisation adds $\lambda \sum_j \beta_j^2$; L1 regularisation adds $\lambda \sum_j |\beta_j|$ and can make some coefficients exactly zero.

What to read in evaluation

- **Accuracy:** overall correctness; can hide minority-class failure.
- **Precision:** among flagged positives, the fraction that is truly positive.
- **Recall:** among actual positives, the fraction detected.
- **F1:** harmonic mean of precision and recall.
- **ROC-AUC:** ranking quality across thresholds.
- **PR-AUC:** often more revealing when the positive class is rare.

Always compare the model with a simple baseline, inspect the confusion matrix, and choose a threshold using a documented validation procedure.\n\n## 🏦 Real-world use cases and dataset\n\nCredit-risk models can support affordability review, portfolio monitoring and early-warning outreach. They must not be used as an unreviewed approval or denial mechanism.\n\nThis project uses the UCI Default of Credit Card Clients dataset. Review the UCI terms and cite the source before redistribution. The target represents next-month default in historical records; it is not a universal definition of creditworthiness.

Step 2 — Install and import the required libraries

In [1]:
```
# pip install -U ipykernel
```

In [2]:
```
# Install the libraries required for our ML project.
# In Google Colab, pandas, NumPy, and scikit-learn
# are usually preinstalled, but this ensures the
# visualization libraries are available as well.

%pip install -q pandas numpy matplotlib seaborn scikit-learn
```

```
Note: you may need to restart the kernel to use updated packages.
```

In [3]:
```
# pandas: load and manipulate tabular datasets.
import pandas as pd

# NumPy: numerical calculations and array operations.
import numpy as np

# Matplotlib: create charts and visualizations.
import matplotlib.pyplot as plt

# Seaborn: create statistical charts.
import seaborn as sns

# train_test_split: separate training data from test data.
from sklearn.model_selection import train_test_split

# Pipeline: combine preprocessing and model training safely.
from sklearn.pipeline import Pipeline

# StandardScaler: standardize numerical features.
from sklearn.preprocessing import StandardScaler
```

```
# LogisticRegression: our classification algorithm.
from sklearn.linear_model import LogisticRegression

# Metrics: measure the performance of our classifier.
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
    ConfusionMatrixDisplay,
    RocCurveDisplay
)

# Display all columns when viewing pandas DataFrames.
pd.set_option("display.max_columns", None)

# Set a consistent chart style for the notebook.
sns.set_theme(style="whitegrid")

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

Step 3 — Load the real dataset and understand its structure

We will use the UCI Credit Card Default dataset. Before training Logistic

Regression, we need to understand what each row represents, what the columns mean, and which column we want to predict. Dataset source: UCI — Default of Credit Card Clients

In [4]:

```
# Import the function used to download the dataset directly
# from the UCI Machine Learning Repository.
from sklearn.datasets import fetch_openml

# Load the credit-card default dataset.
# The data contains historical client records and a
# target indicating default payment in the following month.
credit_data = fetch_openml(
    name="default-of-credit-card-clients",
    version=1,
    as_frame=True
)

# Convert the input features into a pandas DataFrame.
df = credit_data.data.copy()

# Add the target column to the DataFrame.
df["default_payment_next_month"] = credit_data.target

# Display a confirmation message.
print("Dataset loaded successfully.")
```

```
# Display the number of rows and columns.
print("Dataset shape:", df.shape)

# Display the first five records.
display(df.head())
```

```
Dataset loaded successfully.
Dataset shape: (30000, 24)
```

	x1	x2	x3	x4	x5	x6	x7	x8	x9	x10	x11	x12	x13	x14	x15	
0	20000	2	2	1	24	2	2	-1	-1	-2	-2	3913	3102	689	0	
1	120000	2	2	2	26	-1	2	0	0	0	2	2682	1725	2682	3272	
2	90000	2	2	2	34	0	0	0	0	0	0	29239	14027	13559	14331	1
3	50000	2	2	1	37	0	0	0	0	0	0	46990	48233	49291	28314	2
4	50000	1	2	1	57	-1	0	-1	0	0	0	8617	5670	35835	20940	

What this code does

- **fetch_openml**() retrieves the dataset from OpenML.
- **as_frame**=True requests the data in pandas-compatible format.
- **credit_data.data** contains the input features.
- **credit_data.target** contains the target labels.
- **df.shape** returns the number of rows and columns.
- **df.head**() shows the first five records.

In [5]:
```
# 3.2 Check the dataset dimensions


# Count the number of records and columns.
rows, columns = df.shape

print("Total records:", rows)
print("Total columns:", columns)

# Display the column names.
print("\nColumn names:")
print(df.columns.tolist())
```

```
Total records: 30000
Total columns: 24

Column names:
['x1', 'x2', 'x3', 'x4', 'x5', 'x6', 'x7', 'x8', 'x9', 'x10', 'x11', 'x12',
'x13', 'x14', 'x15', 'x16', 'x17', 'x18', 'x19', 'x20', 'x21', 'x22', 'x23',
'default_payment_next_month']
```

In [6]:
```
# 3.3 Understand the column data types

# Display each column's data type and number of non-null values.
# This helps us identify numerical and categorical features
```

```
# and detect columns that may contain missing values.

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 24 columns):
 #   Column                      Non-Null Count  Dtype   
---  ------                      --------------  -----   
 0   x1                          30000 non-null  int64   
 1   x2                          30000 non-null  int64   
 2   x3                          30000 non-null  int64   
 3   x4                          30000 non-null  int64   
 4   x5                          30000 non-null  int64   
 5   x6                          30000 non-null  int64   
 6   x7                          30000 non-null  int64   
 7   x8                          30000 non-null  int64   
 8   x9                          30000 non-null  int64   
 9   x10                         30000 non-null  int64   
 10  x11                         30000 non-null  int64   
 11  x12                         30000 non-null  int64   
 12  x13                         30000 non-null  int64   
 13  x14                         30000 non-null  int64   
 14  x15                         30000 non-null  int64   
 15  x16                         30000 non-null  int64   
 16  x17                         30000 non-null  int64   
 17  x18                         30000 non-null  int64   
 18  x19                         30000 non-null  int64   
 19  x20                         30000 non-null  int64   
 20  x21                         30000 non-null  int64   
 21  x22                         30000 non-null  int64   
 22  x23                         30000 non-null  int64   
 23  default_payment_next_month  30000 non-null  category
dtypes: category(1), int64(23)
memory usage: 5.3 MB
```

Why do we do this?

- Identify numerical and categorical columns.
- Check whether columns contain missing values.
- Understand the dataset structure before preprocessing.

In [7]:

```
# 3.4 Identify the target variable
# Our prediction target is default_payment_next_month.
# - 1 = default payment in the following month.
# - 0 = no default payment in the following month.


# Count the number of observations in each target class.
print(df["default_payment_next_month"].value_counts())

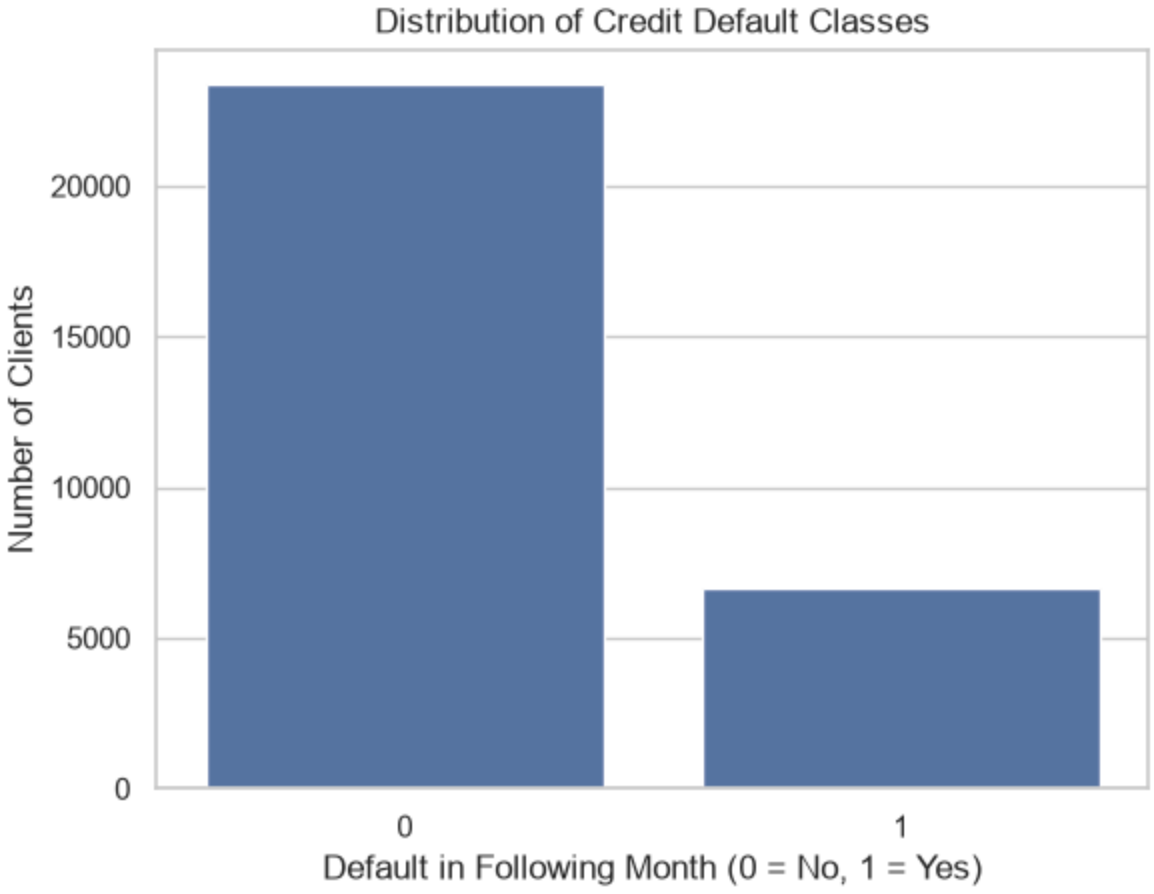
# Calculate the percentage of each class.
print(
    df["default_payment_next_month"]
    .value_counts(normalize=True)
```

```
    .mul(100)
    .round(2)
)

# Visualize the distribution of the target classes.
sns.countplot(
    data=df,
    x="default_payment_next_month"
)

plt.title("Distribution of Credit Default Classes")
plt.xlabel("Default in Following Month (0 = No, 1 = Yes)")
plt.ylabel("Number of Clients")
plt.show()
```

```
default_payment_next_month
0    23364
1     6636
Name: count, dtype: int64
default_payment_next_month
0    77.88
1    22.12
Name: proportion, dtype: float64
```



Why is this important?

- If one class occurs much more frequently than the other, the dataset is imbalanced.
- This affects how we evaluate the model and why accuracy alone may be misleading.

In [8]:
```
# 3.5 Check for missing values and duplicate records


# Count missing values in every column.
print("Missing values per column:")
display(df.isnull().sum().sort_values(ascending=False))

# Count duplicate rows.
print("Duplicate rows:", df.duplicated().sum())
```

```
Missing values per column:
x1                             0
x2                             0
x23                            0
x22                            0
x21                            0
x20                            0
x19                            0
x18                            0
x17                            0
x16                            0
x15                            0
x14                            0
x13                            0
x12                            0
x11                            0
x10                            0
x9                             0
x8                             0
x7                             0
x6                             0
x5                             0
x4                             0
x3                             0
default_payment_next_month     0
dtype: int64
Duplicate rows: 35
```

Step 4 — Exploratory Data Analysis (EDA)

Objective :

- Understand the input features, identify data-quality issues, and examine how the features relate to credit default before training Logistic Regression.

In [9]:
```
# 4.1 Understand the features

# Display the feature names and their data types.
display(df.dtypes.to_frame(name="Data Type"))

# Display summary statistics for numerical columns.
display(df.describe().T)
```

	Data Type
x1	int64
x2	int64
x3	int64
x4	int64
x5	int64
x6	int64
x7	int64
x8	int64
x9	int64
x10	int64
x11	int64
x12	int64
x13	int64
x14	int64
x15	int64
x16	int64
x17	int64
x18	int64
x19	int64
x20	int64
x21	int64
x22	int64
x23	int64
default_payment_next_month	category

	count	mean	std	min	25%	50%	75%
x1	30000.0	167484.322667	129747.661567	10000.0	50000.00	140000.0	240000.00
x2	30000.0	1.603733	0.489129	1.0	1.00	2.0	2.00
x3	30000.0	1.853133	0.790349	0.0	1.00	2.0	2.00
x4	30000.0	1.551867	0.521970	0.0	1.00	2.0	2.00
x5	30000.0	35.485500	9.217904	21.0	28.00	34.0	41.00
x6	30000.0	-0.016700	1.123802	-2.0	-1.00	0.0	0.00
x7	30000.0	-0.133767	1.197186	-2.0	-1.00	0.0	0.00
x8	30000.0	-0.166200	1.196868	-2.0	-1.00	0.0	0.00
x9	30000.0	-0.220667	1.169139	-2.0	-1.00	0.0	0.00
x10	30000.0	-0.266200	1.133187	-2.0	-1.00	0.0	0.00
x11	30000.0	-0.291100	1.149988	-2.0	-1.00	0.0	0.00
x12	30000.0	51223.330900	73635.860576	-165580.0	3558.75	22381.5	67091.00
x13	30000.0	49179.075167	71173.768783	-69777.0	2984.75	21200.0	64006.25
x14	30000.0	47013.154800	69349.387427	-157264.0	2666.25	20088.5	60164.75
x15	30000.0	43262.948967	64332.856134	-170000.0	2326.75	19052.0	54506.00
x16	30000.0	40311.400967	60797.155770	-81334.0	1763.00	18104.5	50190.50
x17	30000.0	38871.760400	59554.107537	-339603.0	1256.00	17071.0	49198.25
x18	30000.0	5663.580500	16563.280354	0.0	1000.00	2100.0	5006.00
x19	30000.0	5921.163500	23040.870402	0.0	833.00	2009.0	5000.00
x20	30000.0	5225.681500	17606.961470	0.0	390.00	1800.0	4505.00
x21	30000.0	4826.076867	15666.159744	0.0	296.00	1500.0	4013.25
x22	30000.0	4799.387633	15278.305679	0.0	252.50	1500.0	4031.50
x23	30000.0	5215.502567	17777.465775	0.0	117.75	1500.0	4000.00

In [10]:
```python
# 4.3 Check feature distributions

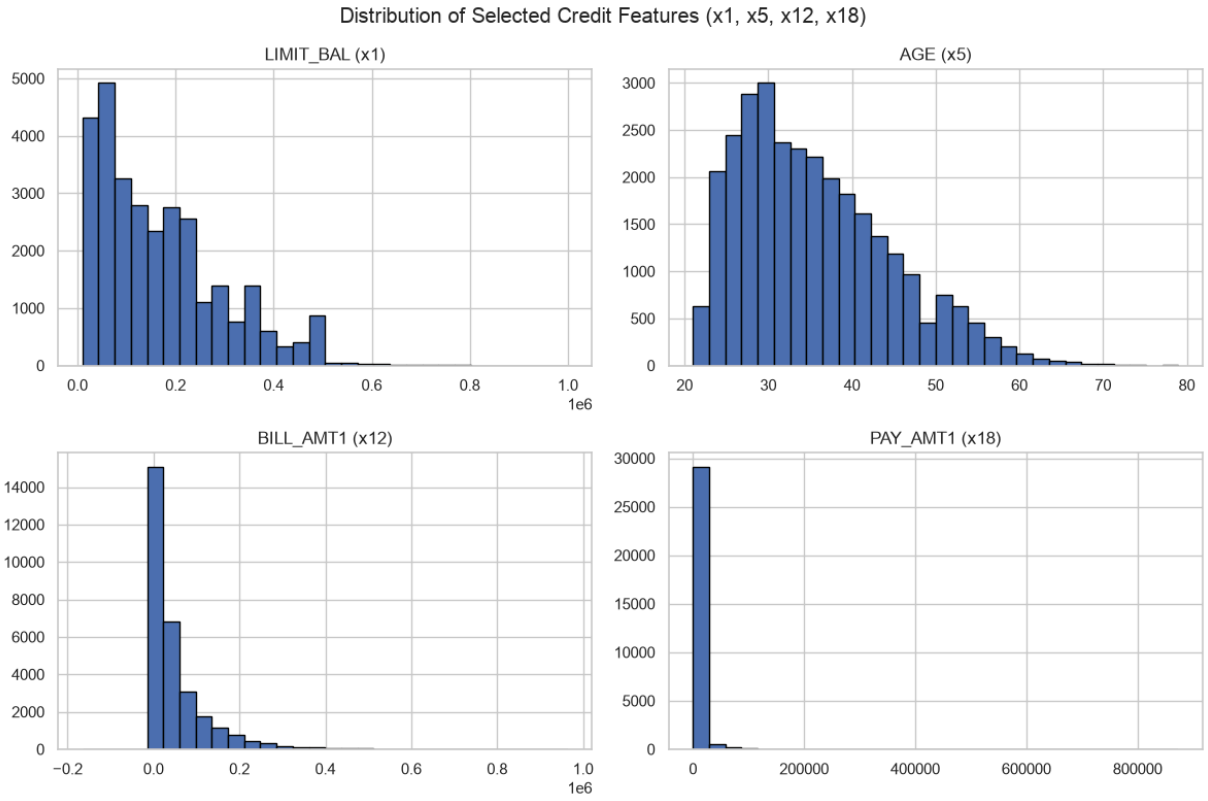
# Select several important numerical features for visualization using their
# x1 = LIMIT_BAL, x5 = AGE, x12 = BILL_AMT1, x18 = PAY_AMT1
selected_features = [
    "x1",
    "x5",
    "x12",
    "x18"
]

# Plot each feature's distribution.
df[selected_features].hist(
```

```
    bins=30,
    figsize=(12, 8),
    edgecolor="black"
)

plt.suptitle("Distribution of Selected Credit Features (x1, x5, x12, x18)")
# Rename the subplot titles to match original names for readability
axes = plt.gcf().get_axes()
feature_names = ["LIMIT_BAL (x1)", "AGE (x5)", "BILL_AMT1 (x12)", "PAY_AMT1
for ax, name in zip(axes, feature_names):
    ax.set_title(name)

plt.tight_layout()
plt.show()
```



In [11]:
```
# 4.4 Compare features with the target

# Compare average values of selected features
# for clients with and without next-month default.
# Using the actual dataset column names:
# x1 = LIMIT_BAL, x5 = AGE, x6 = PAY_0, x12 = BILL_AMT1, x18 = PAY_AMT1
comparison = (
    df.groupby("default_payment_next_month", observed=False)[
        ["x1", "x5", "x6", "x12", "x18"]
    ]
    .mean()
    .T
)

# Rename the index and columns for readability
comparison.index = ["LIMIT_BAL (x1)", "AGE (x5)", "PAY_0 (x6)", "BILL_AMT1 (
```

```
comparison.columns = ["No Default (0)", "Default (1)"]

display(comparison.round(2))
```

	No Default (0)	Default (1)
LIMIT_BAL (x1)	178099.73	130109.66
AGE (x5)	35.42	35.73
PAY_0 (x6)	-0.21	0.67
BILL_AMT1 (x12)	51994.23	48509.16
PAY_AMT1 (x18)	6307.34	3397.04

Why?

- This comparison helps us explore whether feature averages differ between the two target classes.
- A difference in averages does not prove causation, nor does it establish that a feature is independently predictive. We will validate predictive value using held-out data.

In [12]:
```
# 4.5 Examine feature correlations

# Convert the target to numeric form for correlation analysis.
# This is for exploratory analysis only.
eda_df = df.copy()

eda_df["default_payment_next_month"] = pd.to_numeric(
    eda_df["default_payment_next_month"]
)

# Calculate pairwise Pearson correlations between numerical columns.
correlation_matrix = eda_df.corr(numeric_only=True)

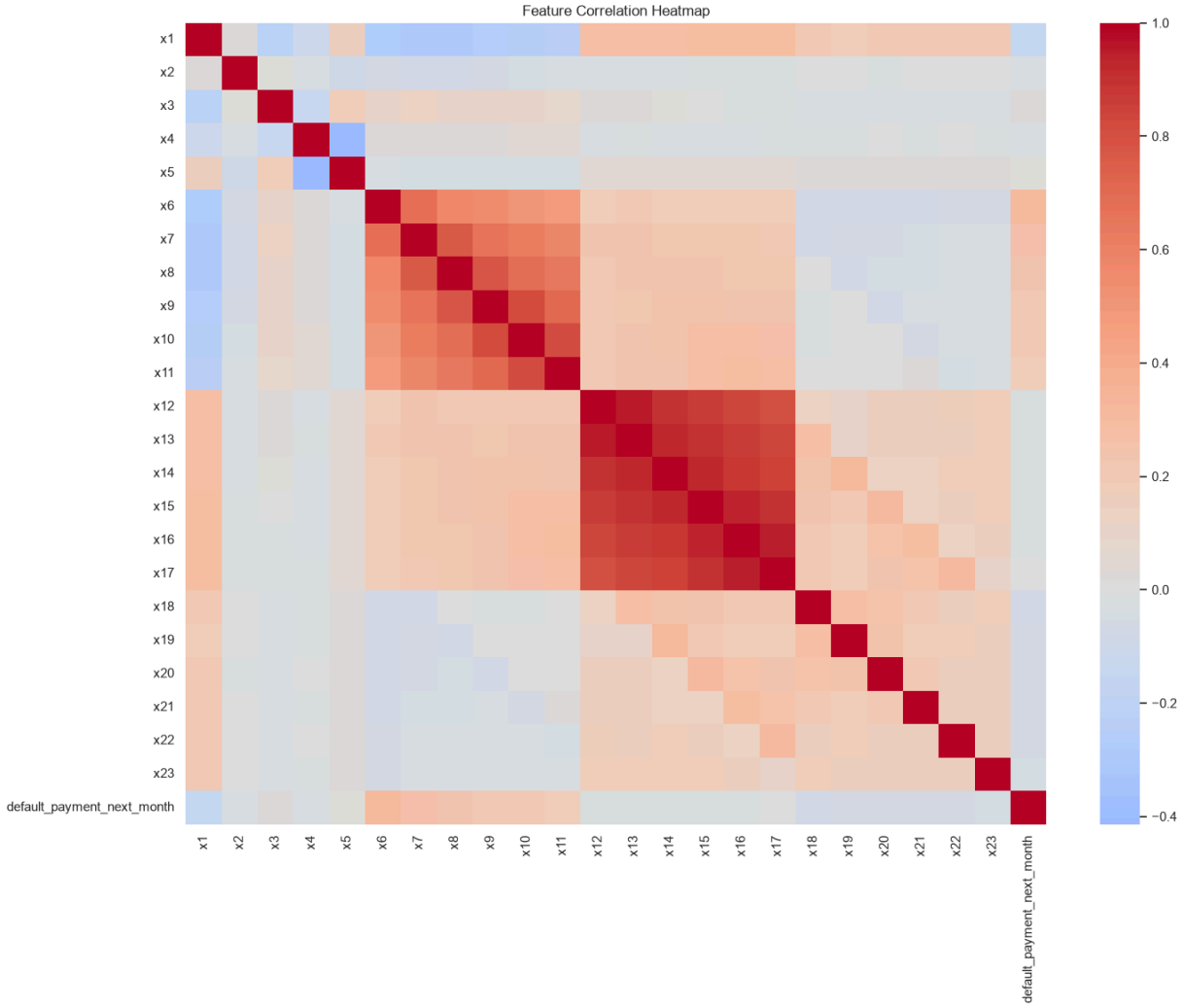
# Display a heatmap of the correlations.
plt.figure(figsize=(16, 12))

sns.heatmap(
    correlation_matrix,
    cmap="coolwarm",
    center=0,
    annot=False
)

plt.title("Feature Correlation Heatmap")
plt.show()

# Show the features with the strongest absolute
# correlation with the target, excluding the target itself.
target_corr = correlation_matrix[
    "default_payment_next_month"
].drop("default_payment_next_month")
```

```
display(
    target_corr.reindex(
        target_corr.abs().sort_values(ascending=False).index
    ).head(10)
)
```




```
x6     0.324794
x7     0.263551
x8     0.235253
x9     0.216614
x10    0.204149
x11    0.186866
x1    -0.153520
x18   -0.072929
x19   -0.058579
x21   -0.056827
Name: default_payment_next_month, dtype: float64
```

Why?

- Correlation helps us identify linear associations between numerical columns.
- Highly correlated predictors can make individual Logistic Regression coefficients harder to interpret.

- Correlation alone does not establish causation or determine which features must be removed.

Step 5 — Data Preprocessing and Train/Test Split

Objective:

- Prepare the dataset for Logistic Regression and ensure that the model is evaluated on data it has not seen during training.

In [13]:
```
# 5.1 Separate features and target

# Define the target: default payment in the following month.
target_column = "default_payment_next_month"

# X contains the input features used for prediction.
X = df.drop(columns=[target_column]).copy()

# y contains the actual target labels.
y = pd.to_numeric(df[target_column]).astype(int)

# Display the dimensions of the features and target.
print("Feature matrix shape:", X.shape)
print("Target vector shape:", y.shape)

# Verify the target classes.
print("\nTarget distribution:")
print(y.value_counts())
```

```
Feature matrix shape: (30000, 23)
Target vector shape: (30000,)

Target distribution:
default_payment_next_month
0    23364
1     6636
Name: count, dtype: int64
```

***Why? ***

- The model must learn from input features X to predict the target y. Including the target in X would cause data leakage.

In [14]:
```
# 5.2 Identify categorical and numerical features using the dataset's actual
# The OpenML version used in this notebook exposes the original features as
# x2, x3, x4 and x6...x11 are encoded categorical/status variables.

categorical_features = [
    "x2",   # SEX
    "x3",   # EDUCATION
```

```
    "x4",   # MARRIAGE
    "x6",   # PAY_0
    "x7",   # PAY_2
    "x8",   # PAY_3
    "x9",   # PAY_4
    "x10",  # PAY_5
    "x11"   # PAY_6
]

numerical_features = [
    column for column in X.columns
    if column not in categorical_features
]

print("Categorical features:", len(categorical_features))
print("Numerical features:", len(numerical_features))
print("Categorical columns:", categorical_features)
```

```
Categorical features: 9
Numerical features: 14
Categorical columns: ['x2', 'x3', 'x4', 'x6', 'x7', 'x8', 'x9', 'x10', 'x1
1']
```

Why?

- One-hot encoding represents categories as separate binary indicators.
- Scaling numerical features helps Logistic Regression optimize its coefficients more effectively.

In [15]:
```
# 5.3 Split the dataset : We will reserve 20% of the observations for testin


# Import the function used to split the dataset.
from sklearn.model_selection import train_test_split

# Split the data before fitting any preprocessing transformations.
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,     # Reserve 20% for final evaluation.
    random_state=42,    # Make the split reproducible.
    stratify=y          # Preserve the target class proportions.
)

# Display the sizes of the resulting datasets.
print("Training features:", X_train.shape)
print("Testing features:", X_test.shape)
print("Training labels:", y_train.shape)
print("Testing labels:", y_test.shape)
```

```
Training features: (24000, 23)
Testing features: (6000, 23)
Training labels: (24000,)
Testing labels: (6000,)
```

Why?

- test_size=0.20: holds out 20% of the data.
- random_state=42: makes the split reproducible.
- stratify=y: helps maintain the class distribution in both subsets. We must not use the test set to fit preprocessing transformations or select model parameters.

In [16]:
```
# 5.4 Build the preprocessing pipeline
# We will scale numerical features and one-hot encode categorical features.

# ColumnTransformer applies different preprocessing
# operations to different groups of features.
from sklearn.compose import ColumnTransformer

# StandardScaler standardizes numerical feature values.
from sklearn.preprocessing import StandardScaler

# OneHotEncoder converts categorical values into
# binary indicator columns.
from sklearn.preprocessing import OneHotEncoder

# Define preprocessing separately for each feature group.
preprocessor = ColumnTransformer(
    transformers=[
        (
            "numerical",
            StandardScaler(),
            numerical_features
        ),
        (
            "categorical",
            OneHotEncoder(
                handle_unknown="ignore"
            ),
            categorical_features
        )
    ]
)

print("Preprocessing pipeline defined.")
```

```
Preprocessing pipeline defined.
```

Why these transformations?

- ***StandardScaler***: transforms numerical features using the training-set mean and standard deviation.
- ***OneHotEncoder***: represents categorical values without imposing an artificial numerical ordering.
- ***ColumnTransformer***: applies each transformation to the correct columns.
- ***Pipeline***: will combine preprocessing and Logistic Regression so the transformations are fitted correctly during training.

handle_unknown="ignore" allows the encoder to process an unfamiliar category at inference time without raising an error.

In [17]:
```
# 5.5 Validate the split and preprocessing setup

# Check that the train and test sets contain both classes.
print("Training class proportions:")
print(y_train.value_counts(normalize=True).round(3))

print("\nTesting class proportions:")
print(y_test.value_counts(normalize=True).round(3))

# Confirm that the feature columns are identical
# in both subsets.
assert list(X_train.columns) == list(X_test.columns)

# Confirm that no target column remains in the features.
assert target_column not in X_train.columns

print("\nTrain/test validation passed.")
```

```
Training class proportions:
default_payment_next_month
0    0.779
1    0.221
Name: proportion, dtype: float64

Testing class proportions:
default_payment_next_month
0    0.779
1    0.221
Name: proportion, dtype: float64

Train/test validation passed.
```

Step 5 summary

We have now:

- Separated the input features from the target.
- Identified categorical and numerical columns.
- Split the data into training and testing subsets.
- Defined the preprocessing transformations.
- Established a leakage-safe structure for model training.

Step 6 — Train the Logistic Regression Model

Objective:

- Train Logistic Regression on the preprocessed training data so it can estimate the probability of credit default.

Formula — Logistic Regression model

$$\hat{p} = \frac{1}{1 + e^{-(w^T x + b)}}$$

In [18]:
```
# Import Pipeline to combine preprocessing and model training.
from sklearn.pipeline import Pipeline

# Import Logistic Regression, our classification algorithm.
from sklearn.linear_model import LogisticRegression

# Create a pipeline with two stages:
# 1. Transform numerical and categorical features.
# 2. Train the Logistic Regression classifier.
model = Pipeline(steps=[
    ("preprocessing", preprocessor),
    ("classifier", LogisticRegression(
        max_iter=1000,  # Allow enough iterations for convergence.
        random_state=42
    ))
])

print("Logistic Regression pipeline created.")
```

```
Logistic Regression pipeline created.
```

Why use a pipeline?

- Ensures training and inference use the same preprocessing.
- Prevents test data from influencing the fitted preprocessing transformations.
- Keeps the workflow reproducible and easier to deploy.

In [19]:
```
# 6.2 Build and train the model using the actual x1...x23 dataset columns.
# Keep these names consistent with X and the deployed Streamlit application.

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

categorical_features = ["x2", "x3", "x4", "x6", "x7", "x8", "x9", "x10", "x1
numerical_features = [column for column in X.columns if column not in categc

preprocessor = ColumnTransformer(
    transformers=[
        ("numerical", StandardScaler(), numerical_features),
```

```
        ("categorical", OneHotEncoder(handle_unknown="ignore"), categorical_
    ]
)

model = Pipeline(steps=[
    ("preprocessing", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000, random_state=42)),
])

model.fit(X_train, y_train)
print("Model training completed successfully.")
print("Training feature order:", list(X_train.columns))
```

```
Model training completed successfully.
Training feature order: ['x1', 'x2', 'x3', 'x4', 'x5', 'x6', 'x7', 'x8', 'x
9', 'x10', 'x11', 'x12', 'x13', 'x14', 'x15', 'x16', 'x17', 'x18', 'x19', 'x
20', 'x21', 'x22', 'x23']
```

What does fit() do?

1. Learns the scaling parameters from numerical training features.
2. Learns the categorical encoding from training features.
3. Estimates Logistic Regression coefficients by optimizing its loss function.

The optimizer seeks coefficients that minimize the regularized logistic loss. Scikit-learn's default Logistic Regression uses L2 regularization unless configured otherwise.

In [20]:
```
# 6.3 Generate predictions

# Predict the class label for each test observation.
y_pred = model.predict(X_test)

# Predict the probability of class 1 (default).
# Column 0 represents class 0; column 1 represents class 1.
y_prob = model.predict_proba(X_test)[:, 1]

# Display a few actual and predicted values.
results = pd.DataFrame({
    "Actual": y_test.to_numpy()[:10],
    "Predicted": y_pred[:10],
    "Default Probability": y_prob[:10]
})

display(results.round(3))
```

	Actual	Predicted	Default Probability
0	0	0	0.156
1	0	0	0.197
2	0	0	0.141
3	1	0	0.097
4	0	0	0.069
5	0	0	0.267
6	0	0	0.114
7	0	0	0.074
8	0	0	0.094
9	0	0	0.111

Why generate both outputs?

- predict() produces class labels using the model's default decision threshold.
- predict_proba() produces estimated probabilities, which let us evaluate alternative thresholds and ranking performance.

In [21]:
```
# 6.4 Check model convergence

# Retrieve the trained Logistic Regression estimator
# from the final pipeline stage.
classifier = model.named_steps["classifier"]

# Display the number of optimization iterations used.
print("Iterations used:", classifier.n_iter_)

# Check whether the maximum iteration limit was reached.
if classifier.n_iter_.max() < classifier.max_iter:
    print("Training finished before the iteration limit.")
else:
    print(
        "Check convergence warnings and solver diagnostics "
        "before proceeding."
    )
```

```
Iterations used: [77]
Training finished before the iteration limit.
```

Step 7 — Evaluate the Logistic Regression Model

Objective:

- Measure how well our trained model predicts credit-card defaults on unseen test data.

In [22]:
```python
# 7.1 Calculate evaluation metrics

# Import the evaluation metrics from scikit-learn.
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

# Calculate accuracy: proportion of all predictions that are correct.
accuracy = accuracy_score(y_test, y_pred)

# Calculate precision: proportion of predicted defaults that are actual defa
precision = precision_score(y_test, y_pred, zero_division=0)

# Calculate recall: proportion of actual defaults detected by the model.
recall = recall_score(y_test, y_pred, zero_division=0)

# Calculate F1-score: harmonic mean of precision and recall.
f1 = f1_score(y_test, y_pred, zero_division=0)

# Calculate ROC-AUC using predicted probabilities, not class labels.
roc_auc = roc_auc_score(y_test, y_prob)

# Display all metrics in a readable format.
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1-score : {f1:.4f}")
print(f"ROC-AUC  : {roc_auc:.4f}")
```

```
Accuracy : 0.8168
Precision: 0.6615
Recall   : 0.3519
F1-score : 0.4594
ROC-AUC  : 0.7583
```

What do these metrics mean?

Metric	Formula	Purpose
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Overall proportion of correct predictions
Precision	$\frac{TP}{TP+FP}$	How many predicted defaults were actual defaults
Recall	$\frac{TP}{TP+FN}$	How many actual defaults were detected
F1-score	$2\frac{PR}{P+R}$	Balance between precision and recall
ROC-AUC	Area under the ROC curve	Measures how well the model ranks positive cases above negative cases

Here, TP means true positives, TN true negatives, FP false positives, and FN false negatives. In our project, the positive class is default.

Enterprise interpretation: A lender may care about identifying likely defaults, but false alarms also have costs. The appropriate balance depends on the business objective and applicable risk controls.

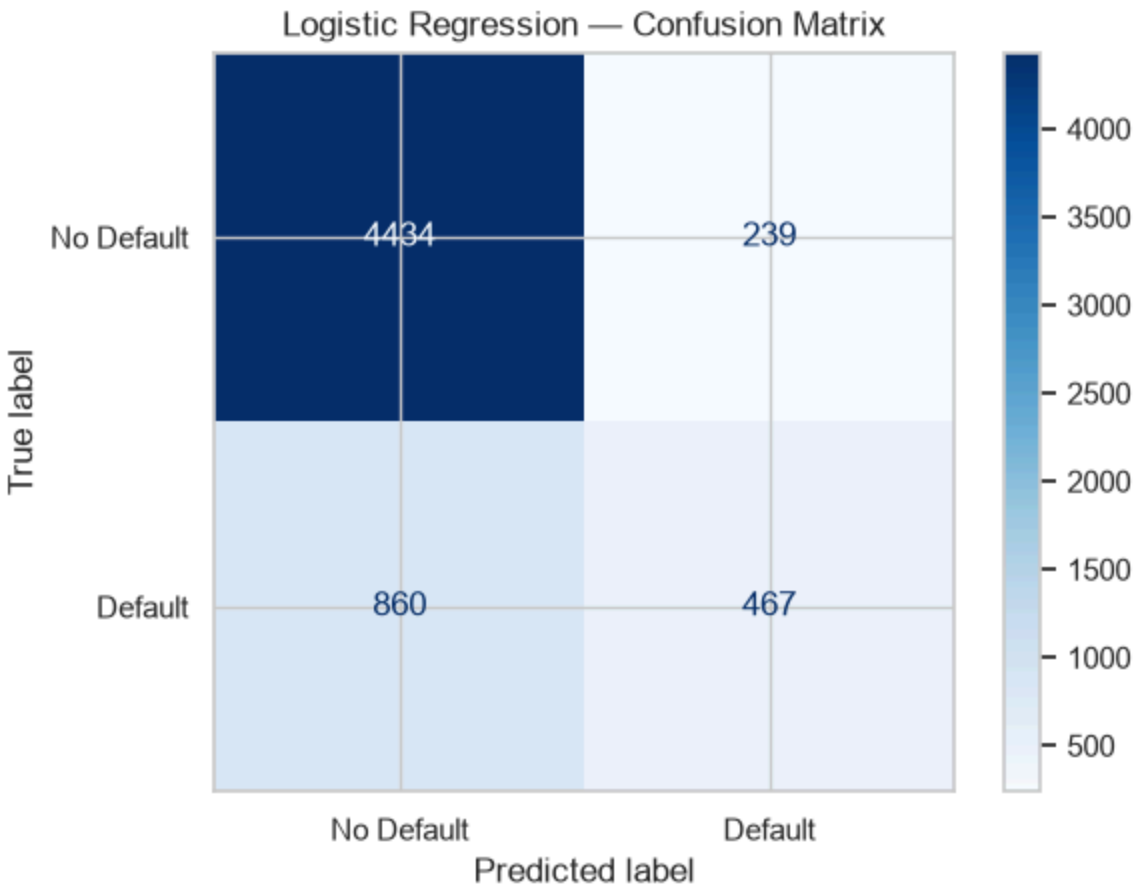
In [23]:
```
# 7.2 Display the confusion matrix

# Import the confusion matrix visualization utility.
from sklearn.metrics import ConfusionMatrixDisplay

# Compare actual labels with predicted labels.
ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred,
    display_labels=["No Default", "Default"],
    cmap="Blues"
)

# Add a title to the visualization.
plt.title("Logistic Regression – Confusion Matrix")

# Display the chart.
plt.show()
```



The ***confusion matrix ***shows four outcomes:

- **True Negative (TN)**: correctly predicted no default.
- **False Positive (FP)**: predicted default, but no default occurred.
- **False Negative (FN)**: predicted no default, but default occurred.
- **True Positive (TP)**: correctly predicted default. False negatives are especially important to analyze when the objective is to detect potential defaults.

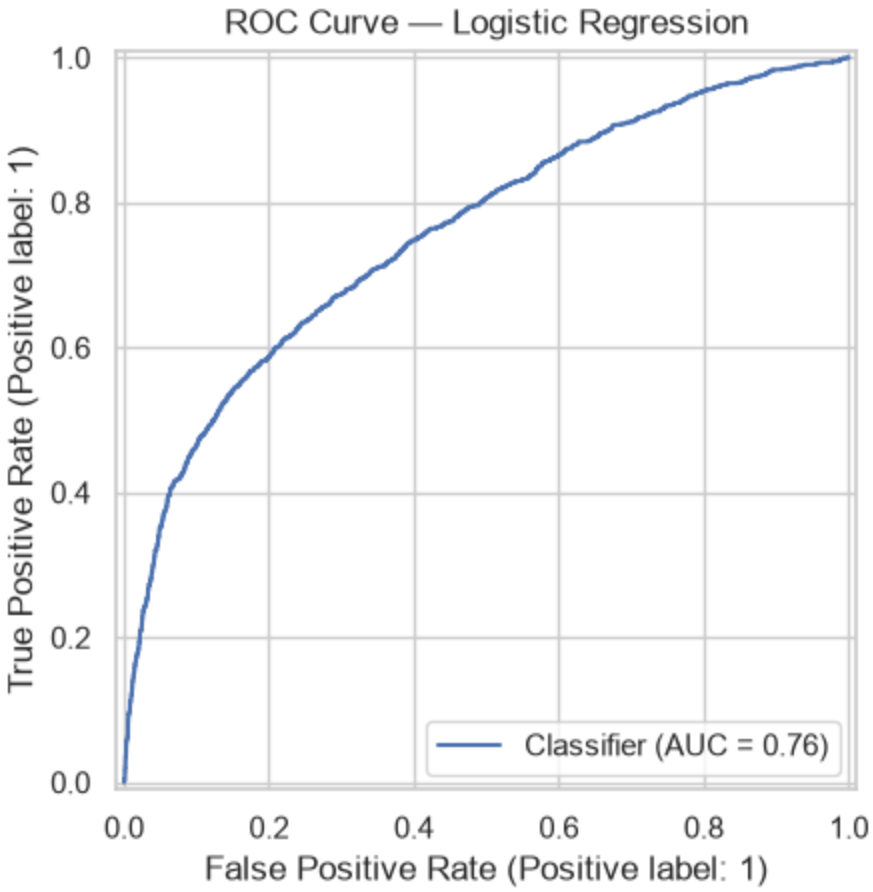
In [24]:
```
# 7.3 Plot the ROC curve


# Import the ROC curve visualization utility.
from sklearn.metrics import RocCurveDisplay

# Plot the ROC curve using actual labels and predicted probabilities.
RocCurveDisplay.from_predictions(y_test, y_prob)

# Add a descriptive title.
plt.title("ROC Curve – Logistic Regression")

# Display the chart.
plt.show()
```



The ***ROC curve*** compares the true positive rate against the false positive rate across different classification thresholds.

- A higher **ROC-AUC** generally indicates better ranking performance. It does not, by itself, establish that the predicted probabilities are well calibrated or that the model is suitable for production.

In [25]:
```
# 7.4 Generate the classification report

# Display precision, recall, F1-score, and sample counts
# separately for each class.
from sklearn.metrics import classification_report

print(
    classification_report(
        y_test,
        y_pred,
        target_names=["No Default", "Default"],
        zero_division=0
    )
)
```

```
              precision    recall  f1-score   support

  No Default       0.84      0.95      0.89      4673
     Default       0.66      0.35      0.46      1327

    accuracy                           0.82      6000
   macro avg       0.75      0.65      0.67      6000
weighted avg       0.80      0.82      0.79      6000
```

This report helps us compare ***performance*** for both classes rather than relying on a single aggregate metric.

Step 8 — Interpret Logistic Regression Coefficients

Objective:

- Understand how input features relate to the model's predicted default probability.

In [26]:
```python
# 8.1 Extract the coefficients

# Get the trained Logistic Regression estimator from the pipeline.
classifier = model.named_steps["classifier"]

# Get the feature names after preprocessing and encoding.
feature_names = model.named_steps["preprocessing"].get_feature_names_out()

# Extract the learned coefficients.
coefficients = classifier.coef_[0]

# Create a table of feature names and their coefficients.
coef_df = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": coefficients
})

# Sort by absolute coefficient magnitude for inspection.
coef_df["Absolute Coefficient"] = coef_df["Coefficient"].abs()
coef_df = coef_df.sort_values(
    "Absolute Coefficient",
    ascending=False
)

# Display the features with the largest coefficient magnitudes.
display(coef_df.head(15).round(4))
```

	Feature	Coefficient	Absolute Coefficient
29	categorical__x6_0	-1.3671	1.3671
27	categorical__x6_-2	-1.1043	1.1043
32	categorical__x6_3	1.0107	1.0107
23	categorical__x4_0	-0.9871	0.9871
31	categorical__x6_2	0.9533	0.9533
21	categorical__x3_5	-0.8590	0.8590
67	categorical__x9_5	-0.7730	0.7730
44	categorical__x7_4	-0.7730	0.7730
76	categorical__x10_4	-0.6553	0.6553
28	categorical__x6_-1	-0.6065	0.6065
83	categorical__x11_0	-0.5199	0.5199
20	categorical__x3_4	-0.4834	0.4834
33	categorical__x6_4	0.4536	0.4536
18	categorical__x3_2	0.4442	0.4442
45	categorical__x7_5	0.4412	0.4412

Why?

- The coefficients are the weights learned by Logistic Regression. Their signs indicate the direction of association with the log-odds of default, holding other model inputs constant.

8.2 Understand the coefficient formula

Logistic Regression models the log-odds of the positive class:

$$\boxed{\log\left(\frac{p}{1-p}\right) = w^T x + b}$$

Where:

- p = predicted probability of default.
- w = vector of learned coefficients.
- x = input feature vector.
- b = intercept.

Why this formula? It connects the model's linear score to the odds of the positive class.

- Positive coefficient: increases the log-odds as the feature increases.
- Negative coefficient: decreases the log-odds as the feature increases.
- Near-zero coefficient: indicates a small conditional linear contribution on the model's scale.

These are associations learned by the model, not proof of causation.

8.3 Convert coefficients into odds ratios

Formula:

$$\boxed{\text{Odds Ratio} = e^{w}}$$

An odds ratio expresses the multiplicative change in the odds of default for a one-unit increase in a feature, holding other inputs constant.

In [27]:
```
# Calculate the odds ratio for each learned coefficient.
coef_df["Odds Ratio"] = np.exp(coef_df["Coefficient"])

# Display coefficients and odds ratios together.
display(
    coef_df[
        ["Feature", "Coefficient", "Odds Ratio"]
    ].head(15).round(4)
)
```

	Feature	Coefficient	Odds Ratio
29	categorical__x6_0	-1.3671	0.2549
27	categorical__x6_-2	-1.1043	0.3315
32	categorical__x6_3	1.0107	2.7476
23	categorical__x4_0	-0.9871	0.3727
31	categorical__x6_2	0.9533	2.5942
21	categorical__x3_5	-0.8590	0.4236
67	categorical__x9_5	-0.7730	0.4616
44	categorical__x7_4	-0.7730	0.4616
76	categorical__x10_4	-0.6553	0.5193
28	categorical__x6_-1	-0.6065	0.5453
83	categorical__x11_0	-0.5199	0.5946
20	categorical__x3_4	-0.4834	0.6167
33	categorical__x6_4	0.4536	1.5740
18	categorical__x3_2	0.4442	1.5593
45	categorical__x7_5	0.4412	1.5545

Interpretation:

- Odds ratio greater than 1: higher feature values are associated with higher odds.
- Odds ratio below 1: higher feature values are associated with lower odds.
- Odds ratio equal to 1: no change in odds for a one-unit increase.

Because our numerical features are standardized, a one-unit increase generally means a one-standard-deviation increase for those features—not one dollar or one year. **One-hot** encoded features require a separate category-aware interpretation.

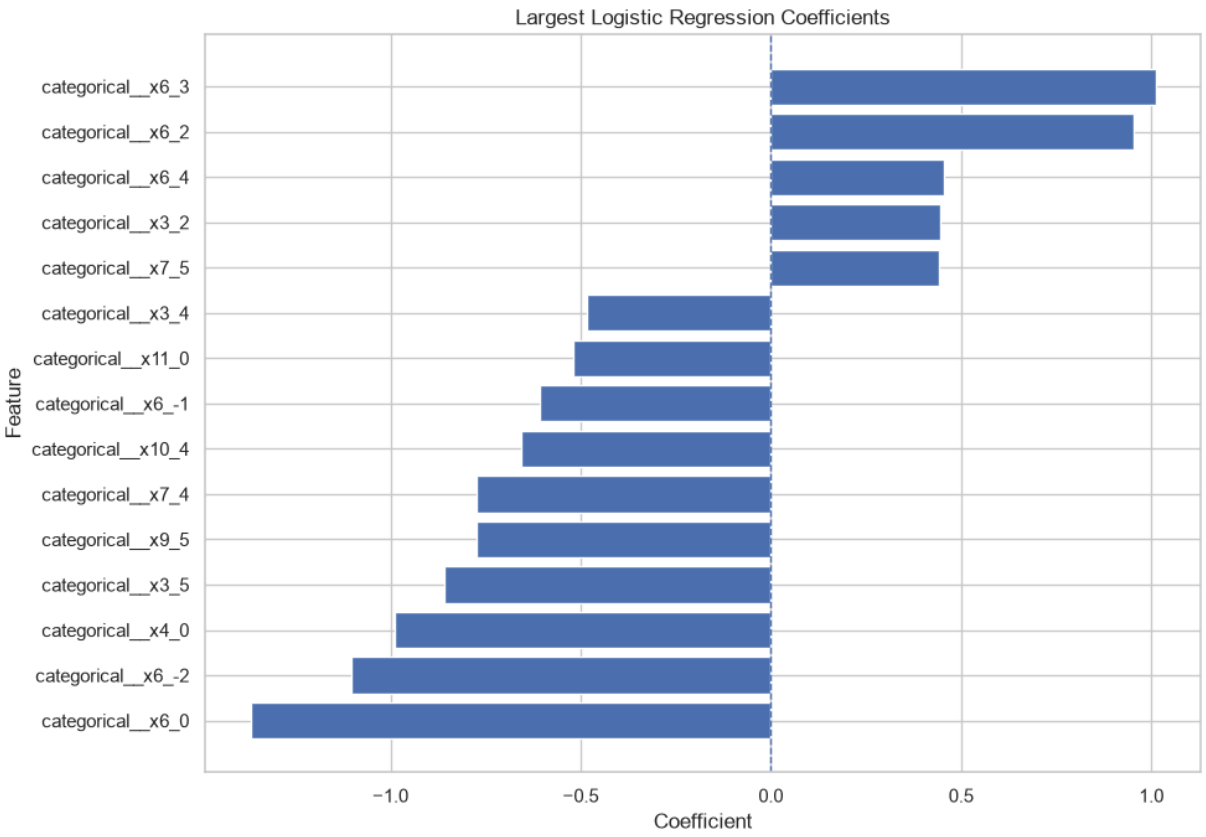
In [28]:
```
# 8.4 Visualize the coefficients
# Select the 15 features with the largest absolute coefficients.
top_features = coef_df.head(15).sort_values("Coefficient")

# Plot positive and negative coefficient values.
plt.figure(figsize=(10, 7))

plt.barh(
    top_features["Feature"],
    top_features["Coefficient"]
)

# The vertical reference at zero separates positive and negative values.
plt.axvline(0, linestyle="--", linewidth=1)
```

```
plt.title("Largest Logistic Regression Coefficients")
plt.xlabel("Coefficient")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```



This chart shows the **direction** and **magnitude** of the selected ***coefficients***.

- Coefficient magnitude alone does not establish feature importance, particularly when features are correlated.

Step 9 — Decision Threshold Tuning

Objective: Understand how changing the classification threshold affects precision and recall.

9.1 Understand the concept

Logistic Regression estimates a probability. The threshold converts that probability into a class.

$$\hat{y} = \begin{cases} 1 & \hat{p} \geq t \\ 0 & \hat{p} < t \end{cases}$$

- Lower threshold → generally more positive predictions and higher recall.
- Higher threshold → generally fewer positive predictions and higher precision.

In [29]:
```
# 9.2 Create a validation set
# We should tune the threshold using validation data, not the test set. This

# Split the existing training data into a smaller training
# set and a validation set for threshold selection.
X_fit, X_val, y_fit, y_val = train_test_split(
    X_train,
    y_train,
    test_size=0.20,
    random_state=42,
    stratify=y_train
)

# Create a separate pipeline for threshold experiments.
threshold_model = Pipeline(steps=[
    ("preprocessing", preprocessor),
    ("classifier", LogisticRegression(
        max_iter=1000,
        random_state=42
    ))
])

# Train only on the smaller training subset.
threshold_model.fit(X_fit, y_fit)

# Estimate default probabilities on validation data.
val_probabilities = threshold_model.predict_proba(X_val)[:, 1]

print("Validation probabilities generated.")
```

```
Validation probabilities generated.
```

In [30]:
```
# 9.3 Compare different thresholds

# Import the metrics needed to compare thresholds.
from sklearn.metrics import precision_score, recall_score, f1_score

# Define candidate thresholds for experimentation.
thresholds = [0.20, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80]
```

```
results = []

for threshold in thresholds:
    # Convert probabilities into binary class predictions.
    val_predictions = (
        val_probabilities >= threshold
    ).astype(int)

    # Calculate the metrics at this threshold.
    results.append({
        "Threshold": threshold,
        "Precision": precision_score(
            y_val, val_predictions, zero_division=0
        ),
        "Recall": recall_score(
            y_val, val_predictions, zero_division=0
        ),
        "F1-score": f1_score(
            y_val, val_predictions, zero_division=0
        )
    })

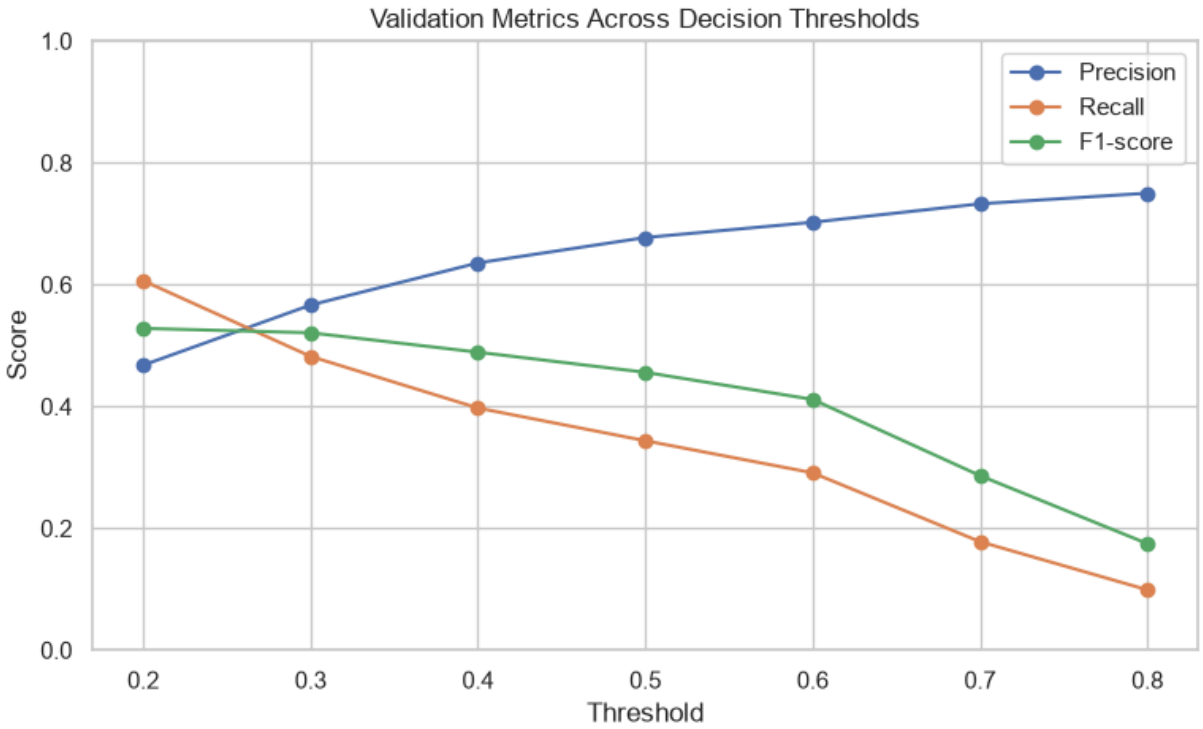
# Display the comparison table.
threshold_results = pd.DataFrame(results)
display(threshold_results.round(3))
```

	Threshold	Precision	Recall	F1-score
0	0.2	0.468	0.606	0.528
1	0.3	0.566	0.482	0.521
2	0.4	0.636	0.397	0.489
3	0.5	0.677	0.344	0.456
4	0.6	0.702	0.291	0.411
5	0.7	0.733	0.178	0.286
6	0.8	0.750	0.099	0.175

In [31]:
```
# 9.4 Visualize the results

# Plot how precision, recall, and F1-score vary with threshold.
threshold_results.set_index("Threshold")[
    ["Precision", "Recall", "F1-score"]
].plot(marker="o", figsize=(9, 5))

plt.title("Validation Metrics Across Decision Thresholds")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.grid(True)
plt.show()
```



Important:

- We should not automatically choose the **threshold** with the highest F1-score.
- In an enterprise setting, **threshold** selection should reflect business costs, acceptable false-positive rates, recall requirements, and any applicable policy constraints.
- For this project, we will choose a **threshold** based on an explicitly documented validation criterion. We will then retrain on the full training set and evaluate once on the held-out test set.

Step 10 — Finalize and Save the Logistic Regression Model

Objective:

- Select the decision threshold using validation data, retrain the model on the complete training set, evaluate it on the untouched test set, and save the complete preprocessing-and-model pipeline.

In [32]:
```
# 10.1 Select the threshold

# Identify the row with the highest validation F1-score.
best_row = threshold_results.loc[
    threshold_results["F1-score"].idxmax()
]
```

```
# Store the selected decision threshold.
best_threshold = float(best_row["Threshold"])

# Display the selected threshold and validation metrics.
print("Selected threshold:", best_threshold)
print("Validation precision:", round(best_row["Precision"], 3))
print("Validation recall:", round(best_row["Recall"], 3))
print("Validation F1-score:", round(best_row["F1-score"], 3))
```

```
Selected threshold: 0.2
Validation precision: 0.468
Validation recall: 0.606
Validation F1-score: 0.528
```

Why?

- We select the threshold using validation data rather than the test set to avoid tuning our decision rule against the final evaluation results.

In [33]:
```
# 10.2 Retrain on the complete training set

# Create the final pipeline using the same preprocessing
# and Logistic Regression configuration.
final_model = Pipeline(steps=[
    ("preprocessing", preprocessor),
    ("classifier", LogisticRegression(
        max_iter=1000,
        random_state=42
    ))
])

# Fit the final pipeline on all training observations.
# The test set remains untouched.
final_model.fit(X_train, y_train)

print("Final model trained successfully.")
```

```
Final model trained successfully.
```

Why?

- After selecting the threshold, we use the full training set to estimate the final model parameters. The preprocessing transformations are also fitted again on the full training data.

In [34]:
```
# 10.3 Evaluate on the test set

# Estimate the probability of default for test observations.
test_probabilities = final_model.predict_proba(X_test)[:, 1]

# Apply the threshold selected on validation data.
test_predictions = (
    test_probabilities >= best_threshold
).astype(int)
```

```
# Calculate final test metrics.
print("Final test evaluation")
print("---------------------")
print("Accuracy:", round(
    accuracy_score(y_test, test_predictions), 3
))
print("Precision:", round(
    precision_score(y_test, test_predictions, zero_division=0), 3
))
print("Recall:", round(
    recall_score(y_test, test_predictions, zero_division=0), 3
))
print("F1-score:", round(
    f1_score(y_test, test_predictions, zero_division=0), 3
))
print("ROC-AUC:", round(
    roc_auc_score(y_test, test_probabilities), 3
))
```

```
Final test evaluation
---------------------
Accuracy: 0.752
Precision: 0.454
Recall: 0.589
F1-score: 0.513
ROC-AUC: 0.758
```

10.4 Save the complete pipeline

In [35]:

```
# 10.4 Save the complete model pipeline, threshold, and UI validation metada
# Run this cell after final_model and best_threshold have been created.

import json
import joblib

# Keep the raw feature names required by the fitted pipeline.
feature_names = list(X_train.columns)

# Human-readable mapping for the original Taiwan credit-card dataset.
feature_schema = {
    "x1":  {"label": "Credit limit (LIMIT_BAL)", "description": "Assigned cr
    "x2":  {"label": "Sex (SEX)", "description": "Dataset category code: 1 c
    "x3":  {"label": "Education (EDUCATION)", "description": "Encoded educat
    "x4":  {"label": "Marital status (MARRIAGE)", "description": "Encoded ma
    "x5":  {"label": "Age (AGE)", "description": "Age recorded in years.", "
    "x6":  {"label": "Most recent repayment status (PAY_0)", "description":
    "x7":  {"label": "Repayment status — 2 months ago (PAY_2)", "description
    "x8":  {"label": "Repayment status — 3 months ago (PAY_3)", "description
    "x9":  {"label": "Repayment status — 4 months ago (PAY_4)", "description
    "x10": {"label": "Repayment status — 5 months ago (PAY_5)", "description
    "x11": {"label": "Repayment status — 6 months ago (PAY_6)", "description
    "x12": {"label": "Bill statement — month 1 (BILL_AMT1)", "description":
    "x13": {"label": "Bill statement — month 2 (BILL_AMT2)", "description":
    "x14": {"label": "Bill statement — month 3 (BILL_AMT3)", "description":
    "x15": {"label": "Bill statement — month 4 (BILL_AMT4)", "description":
```

```
    "x16": {"label": "Bill statement — month 5 (BILL_AMT5)", "description":
    "x17": {"label": "Bill statement — month 6 (BILL_AMT6)", "description":
    "x18": {"label": "Payment amount — month 1 (PAY_AMT1)", "description": "
    "x19": {"label": "Payment amount — month 2 (PAY_AMT2)", "description": "
    "x20": {"label": "Payment amount — month 3 (PAY_AMT3)", "description": "
    "x21": {"label": "Payment amount — month 4 (PAY_AMT4)", "description": "
    "x22": {"label": "Payment amount — month 5 (PAY_AMT5)", "description": "
    "x23": {"label": "Payment amount — month 6 (PAY_AMT6)", "description": "
}

metadata = {
    "model_name": "Credit Card Default Prediction",
    "model_version": "1.0.0",
    "target_name": "default_payment_next_month",
    "positive_class": 1,
    "threshold": float(best_threshold),
    "feature_names": feature_names,
    "categorical_features": categorical_features,
    "features": {}
}

for feature in feature_names:
    series = X_train[feature].dropna()
    entry = dict(feature_schema.get(feature, {
        "label": feature, "description": "Input feature from the training da
        "kind": "numeric", "section": "Other features"
    }))
    entry["min"] = float(series.min())
    entry["max"] = float(series.max())
    entry["median"] = float(series.median())
    if feature in categorical_features:
        entry["categories"] = sorted([int(v) if float(v).is_integer() else f
    metadata["features"][feature] = entry

from pathlib import Path

ARTIFACT_DIR = Path("artifacts")
ARTIFACT_DIR.mkdir(exist_ok=True)
joblib.dump(final_model, ARTIFACT_DIR / "logistic_regression_pipeline.joblib
joblib.dump(float(best_threshold), ARTIFACT_DIR / "decision_threshold.joblib
with (ARTIFACT_DIR / "model_metadata.json").open("w", encoding="utf-8") as f
    json.dump(metadata, file, indent=2, ensure_ascii=False)

print("Saved:", ARTIFACT_DIR / "logistic_regression_pipeline.joblib")
print("Saved:", ARTIFACT_DIR / "decision_threshold.joblib")
print("Saved:", ARTIFACT_DIR / "model_metadata.json")
print("Feature count:", len(feature_names))
```

```
Saved: artifacts/logistic_regression_pipeline.joblib
Saved: artifacts/decision_threshold.joblib
Saved: artifacts/model_metadata.json
Feature count: 23
```

Why save the pipeline?

- It preserves the fitted encoders, scaler, and Logistic Regression model together.

- The deployed application can then apply exactly the same transformations used during training.
- `model_metadata.json` contains feature labels, descriptions, categories, and training-set ranges for the UI.

10.5 Download the model artifacts from Colab

In [36]:
```
# Download artifacts in Colab; report their locations in local Jupyter.
from pathlib import Path

artifact_files = [
    Path("artifacts/logistic_regression_pipeline.joblib"),
    Path("artifacts/decision_threshold.joblib"),
    Path("artifacts/model_metadata.json"),
]

try:
    from google.colab import files
except ImportError:
    print("Artifacts saved locally:")
    for artifact_file in artifact_files:
        print(artifact_file.resolve())
else:
    for artifact_file in artifact_files:
        files.download(str(artifact_file))
```

```
Artifacts saved locally:
/Users/abhishekpanda/Documents/machine-learning-algorithms/Supervised-ML/Classification/Logistic-Regression/01-credit-card-default/artifacts/logistic_regression_pipeline.joblib
/Users/abhishekpanda/Documents/machine-learning-algorithms/Supervised-ML/Classification/Logistic-Regression/01-credit-card-default/artifacts/decision_threshold.joblib
/Users/abhishekpanda/Documents/machine-learning-algorithms/Supervised-ML/Classification/Logistic-Regression/01-credit-card-default/artifacts/model_metadata.json
```

Step 11 — Finalize and Validate the Google Colab Notebook

Objective:

- Ensure our Logistic Regression notebook is reproducible, properly documented, and ready to publish on GitHub.
- We will keep the complete model-development workflow in one notebook: `logistic_regression.ipynb`.

Final Project Summary

Problem

Predict next-month credit-card default using historical client data.

Dataset

UCI Default of Credit Card Clients dataset.

Algorithm

Logistic Regression — Binary Classification.

Workflow

1. Data loading and exploration
2. Exploratory Data Analysis
3. Feature and target separation
4. Train/validation/test split
5. Feature preprocessing
6. Logistic Regression training
7. Model evaluation
8. Coefficient interpretation
9. Decision threshold selection
10. Final model training and serialization

Evaluation

The final test metrics are calculated in the notebook. No performance values are assumed in advance.

Limitations

- Historical associations do not establish causation.
- Performance may change on new populations or over time.
- Threshold selection depends on business objectives.
- This educational model is not approved for real lending decisions.

Reproducibility

The notebook uses fixed random seeds and a preprocessing pipeline.

In [37]:
```python
# 11.2 Verify the final model and threshold


# Confirm that the final pipeline contains both required stages.
assert "preprocessing" in final_model.named_steps
assert "classifier" in final_model.named_steps

# Confirm that the threshold is within the valid probability range.
assert 0 <= best_threshold <= 1

# Confirm that the model can produce probabilities for test records.
probability_check = final_model.predict_proba(X_test.iloc[:5])

# Each row should contain probabilities for both binary classes.
assert probability_check.shape == (5, 2)

# Confirm that each pair of probabilities sums approximately to 1.
assert np.allclose(probability_check.sum(axis=1), 1.0)

print("Model validation checks passed.")
```

```
Model validation checks passed.
```

In [38]:
```python
# Print the Python and library versions used to train the model.
import sys
import sklearn
import numpy
import scipy
import pandas
import joblib

print("Python:", sys.version)
print("scikit-learn:", sklearn.__version__)
print("NumPy:", numpy.__version__)
print("SciPy:", scipy.__version__)
print("Pandas:", pandas.__version__)
print("Joblib:", joblib.__version__)
```

```
Python: 3.12.13 (main, Jul 23 2026, 14:33:47) [Clang 22.1.3 ]
scikit-learn: 1.6.1
NumPy: 2.1.3
SciPy: 1.16.3
Pandas: 2.3.3
Joblib: 1.6.0
```

In [39]:
```python
import sklearn
import sys

print("Python version:", sys.version)
print("scikit-learn version:", sklearn.__version__)
```

```
Python version: 3.12.13 (main, Jul 23 2026, 14:33:47) [Clang 22.1.3 ]
scikit-learn version: 1.6.1
```

In [40]:
```python
import numpy as np
import scipy
import joblib
```

```
print("NumPy:", np.__version__)
print("SciPy:", scipy.__version__)
print("joblib:", joblib.__version__)
```

```
NumPy: 2.1.3
SciPy: 1.16.3
joblib: 1.6.0
```

✅ Book-ready summary and checklist

Key ideas

- Logistic Regression models a linear relationship in log-odds and returns calibrated-looking probabilities only when the data and calibration procedure support that interpretation.
- The sigmoid maps a score to a probability; the threshold maps a probability to an action.
- Cross-entropy rewards accurate probabilities and strongly penalises confident mistakes.
- Coefficients describe conditional associations. They do not establish causality or fairness.
- Preprocessing must be fitted on training data only, and the complete pipeline should be saved for serving.

Practical checklist

- [x] Problem, target and intended use are stated.
- [x] Dataset source and access/licensing notes are documented.
- [x] Train/validation/test logic and leakage controls are explained.
- [x] Logistic function, log-odds and cross-entropy formulas are included.
- [x] Baseline, confusion matrix and threshold-aware metrics are reviewed.
- [x] Coefficients are interpreted cautiously.
- [x] Saved artifacts are reloaded with the pinned dependency versions.
- [ ] Validate on a current, approved dataset and monitor drift.
- [ ] Complete privacy, fairness, governance and security review.
- [x] Streamlit profile and app directory: https://share.streamlit.io/user/abhishekpandaofficial.

Libraries and tools

Python 3.12 · Jupyter · pandas · NumPy · Matplotlib · Seaborn · scikit-learn · joblib · openpyxl · nbconvert · Streamlit · Git/GitHub.

🚀 Streamlit



Browse the deployed project apps from the Streamlit profile and app directory.